0000 0101
1110 0111
0x0FFF

42 MULHOUSE

# Les bases du C

*... ou pas*

Ce qui se passe vraiment dans la mémoire

LE BINAIRE

# Le binaire sur 8 bits

Compter en binaire

| Décimal | Binaire |
|---|---|
| 0 | 0000 0000 |
| 1 | 0000 0001 |
| 2 | 0000 0010 |
| 3 | 0000 0011 |
| 4 | 0000 0100 |
| 5 | 0000 0101 |

Chaque position vaut une puissance de 2

| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 |

`0001 0100` = 16 + 4 = 20

On additionne les poids des bits à 1. Un octet = 8 bits, de 0 à 255.

CALCULER EN BINAIRE

# Les négatifs : le complément à deux

Exemples sur 8 bits (un char signé) pour rester lisible ; sur un int, c'est la même chose avec 32 bits.

### 1 Écrire la valeur positive

`0001 1001`

25

### 2 Inverser tous les bits (~)

`1110 0110`

~25

### 3 Ajouter 1

`1110 0111`

−25

### Le bit de gauche vaut 1 pour tout négatif…

… mais ce n'est pas un simple signe : −1 s'écrit 1111 1111, pas 1000 0001.

### Les plages

char : −128 → 127
int : −2 147 483 648 → 2 147 483 647

CALCULER EN BINAIRE

# Additionner avec des négatifs

10 + (−25)

```
  0000 1010   (10)
+ 1110 0111   (-25)
= 1111 0001   (-15)
```

−1 + (−1)

```
    1111 1111   (-1)
+   1111 1111   (-1)
= 1 1111 1110
→   1111 1110   (-2)
```

Le 1 qui sort à gauche est ignoré.

### Retenue ≠ overflow

Le bit qui sort est une **retenue** (carry) : il est ignoré et le résultat reste juste. Un **overflow**, c'est quand le vrai résultat ne tient pas dans le type, comme INT_MAX + 1. En C, l'overflow d'un entier signé est un comportement indéfini.

En binaire : 0 + 0 = 0 · 0 + 1 = 1 · 1 + 1 = 0, je retiens 1

CALCULER EN BINAIRE

# Multiplier : décaler et additionner

Décaler de n bits vers la gauche multiplie par $2^n$

$$x \ll n = x \times 2^n$$

$3 \times 3 = 3 \times (2 + 1) = (3 \ll 1) + 3$

**Le calcul**

```
  0000 0110   (3 << 1 = 6)
+ 0000 0011   (3)
= 0000 1001   (9)
```

### Le processeur sait multiplier

Il a une instruction pour ça (imul sur x86). C'est le compilateur qui remplace parfois x * 8 par x << 3 quand c'est plus rapide. Décaler puis additionner, c'est le principe, pas la seule façon de faire.

CALCULER EN BINAIRE

# Diviser : décaler à droite

Décaler de n bits vers la droite divise par $2^n$

$$x >> n = x / 2^n$$

Division entière : le reste est jeté, comme avec / en C.

**4 >> 1**

```
  0000 0100   (4)
>> 1
= 0000 0010   (2)
```

**5 >> 1**

```
  0000 0101   (5)
>> 1            → 1 sort
= 0000 0010   (2)
```

### Le bit qui sort, c'est le reste

5 / 2 vaut aussi 2 en C. Le bit perdu vaut 1 : c'est 5 % 2, qu'on peut lire directement avec 5 & 1.

### Et diviser par 3 ?

Pas de décalage possible : le processeur a une instruction de division (idiv). Attention : −5 >> 1 donne −3 alors que −5 / 2 donne −2.

LES OPÉRATEURS BIT À BIT

# Le poids des bits : & et |

**ET : &**

```
  0000 1010   (10)
& 0000 0001   (1)
= 0000 0000   (0)
```

Garde un bit seulement s'il vaut 1 des deux côtés. n & 1 donne le bit de poids 1 : 0 = pair, 1 = impair.

**OU : |**

```
  0000 1010   (10)
| 0000 0001   (1)
= 0000 1011   (11)
```

Met le bit à 1 s'il vaut 1 d'un côté ou de l'autre.

| a | b | a & b | a \| b |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 |

**Petit test :** que vaut 10 & −1 ? (indice : comment s'écrit −1 ?)

LES OPÉRATEURS BIT À BIT

# Échanger les deux moitiés d'un int

```
unsigned int n     = 0x12345678;
unsigned int left  = n << 16;      // 0x56780000
unsigned int right = n >> 16;      // 0x00001234
unsigned int rslt  = left | right; // 0x56781234
```

### 12 34 | 56 78 → 56 78 | 12 34

On échange les deux moitiés de 16 bits : une moitié part à gauche, l'autre à droite, et | les recolle.

### Toujours en unsigned

Sur un int signé, décaler dans le bit de signe est un comportement indéfini, et >> peut recopier le bit de signe.

LA MÉMOIRE

# Les types

| Type | Taille | Min (signé) | Max (signé) | Max (non signé) |
|---|---|---|---|---|
| char | 1 | -128 | 127 | 255 |
| short | 2 | -32 768 | 32 767 | 65 535 |
| int | 4 | -2 147 483 648 | 2 147 483 647 | 4 294 967 295 |

### Pourquoi un max plus petit en signé ?

Un bit sert au signe : il reste 7 bits pour la valeur sur un char, d'où 127 au lieu de 255.

### sizeof(type)

Donne la taille en octets. Indispensable avec malloc : malloc(n * sizeof(int)).

Tailles sur nos machines (Linux 64 bits) : la norme du C ne garantit que des minimums. long, float, double : pour une autre fois.

LA MÉMOIRE

# int nbr = 5

| Adresse | Octet |
|---|---|
| 0x0FFF | ???? ???? |
| 0x1000 | 0000 0101 |
| 0x1001 | 0000 0000 |
| 0x1002 | 0000 0000 |
| 0x1003 | 0000 0000 |
| 0x1004 | ???? ???? |

### Un int = 4 octets

int nbr = 5; réserve 0x1000 à 0x1003.

### Little-endian

Nos processeurs rangent l'octet de poids faible en premier, à la plus petite adresse. Le bit de signe est donc dans l'octet 0x1003.

Relu dans l'ordre des poids :

`0000 0000 | 0000 0000 | 0000 0000 | 0000 0101`

= +5

LA MÉMOIRE

# Réassigner : nbr = 6

Avant

| Adresse | Octet |
|---|---|
| 0x0FFF | ???? ???? |
| 0x1000 | 0000 0101 |
| 0x1001 | 0000 0000 |
| 0x1002 | 0000 0000 |
| 0x1003 | 0000 0000 |
| 0x1004 | ???? ???? |

→

Après nbr = 6;

| Adresse | Octet |
|---|---|
| 0x0FFF | ???? ???? |
| 0x1000 | 0000 0110 |
| 0x1001 | 0000 0000 |
| 0x1002 | 0000 0000 |
| 0x1003 | 0000 0000 |
| 0x1004 | ???? ???? |

```
int nbr = 5; // réserve 0x1000 à 0x1003 et y écrit 5
nbr = 6;     // réécrit l'octet 0x1000 : 0101 devient 0110
```

LA MÉMOIRE

# char c = ’a’

| Adresse | Octet | ASCII | Car. |
|---|---|---|---|
| 0x0FFF | ???? ???? | | |
| 0x1000 | 0110 0001 | 97 | a |
| 0x1001 | ???? ???? | | |

Il n’y a que des 0 et des 1. Une lettre est donc rangée sous forme de nombre : son code **ASCII**, sur un seul octet.

```
'a' = 97 = 0110 0001
```

### Conséquence

Un char est un petit nombre : 'a' + 1 == 'b' et 'A' + 32 == 'a'.

LES POINTEURS

# Les pointeurs : & et *

```c
int a = 42;
int *ptr = &a;       // ptr contient l'adresse de a
printf("%d", *ptr); // affiche 42
*ptr = 21;           // modifie a
```

| Écriture | Sens |
|---|---|
| int *ptr | ptr est un pointeur sur int |
| &a | l'adresse de a |
| *ptr | la valeur à l'adresse contenue dans ptr |




Un pointeur est une variable comme une autre : sa valeur est une adresse.

LES POINTEURS

# char *str = ”chat”

la variable

`str`
`0x1000`

8 octets

→

la chaîne (5 octets)

| Adresse | Octet | ASCII | Car. |
|---|---|---|---|
| 0x0FFF | ???? ???? | | |
| 0x1000 | 0110 0011 | 99 | c |
| 0x1001 | 0110 1000 | 104 | h |
| 0x1002 | 0110 0001 | 97 | a |
| 0x1003 | 0111 0100 | 116 | t |
| 0x1004 | 0000 0000 | 0 | \0 |
| 0x1005 | ???? ???? | | |

### `sizeof(str) == 8`

La taille du pointeur (64 bits), pas celle de la chaîne.

### `strlen(str) == 4`

La chaîne occupe 5 octets en comptant le \0.

### `lecture seule`

« chat » est un littéral : str[0] = 'C' fait planter le programme. Copie modifiable : char arr[] = "chat";

LES POINTEURS

# char **strs : un tableau de pointeurs




### strs[i] choisit le mot, strs[i][j] la lettre

strs[1] vaut 0x2000, donc strs[1][0] vaut 'l'.

### Le tableau se termine par NULL

Pas par une chaîne vide « \0 ». C'est exactement ce que doit renvoyer ft_split.

STACK ET HEAP

# Stack et heap

## Stack

- gérée automatiquement : variables locales
- taille limitée (souvent 8 Mo sous Linux)
- libérée à la sortie de la fonction

```
char str[5] = "chat";
```

## Heap

- gérée par toi : malloc et free
- limitée par la RAM disponible
- à libérer, sinon fuite mémoire

```
char *str;

str = malloc(5 * sizeof(char));
if (!str)
    return (NULL);
strcpy(str, "chat");
free(str);
```

Ne jamais renvoyer l'adresse d'une variable locale : elle disparaît avec la fonction. Et toujours vérifier le retour de malloc.

STACK ET HEAP

# La mémoire en pages



1 case = 1 page de 4096 octets

pleines = données à ton programme

### Le système distribue des pages

La mémoire est donnée par pages de 4096 octets, entières, même si tu n'en utilises qu'un octet.

### Les pages neuves sont à zéro

Par sécurité, le noyau efface une page avant de te la donner : tu ne vois jamais les données d'un autre programme.

### Alors d'où vient le « n'importe quoi » ?

De ton propre programme : anciennes variables de la stack, blocs déjà libérés avec free. D'où l'importance d'initialiser.

STACK ET HEAP

# Comment malloc découpe la mémoire

Modèle simplifié : malloc demande de grands blocs au système, puis les découpe lui-même.

### malloc(100)




Un morceau du bloc, précédé d'un petit en-tête caché (H).

### malloc(200)




Le reste du bloc sert aux malloc suivants, sans redemander au système.

### Plus de place




malloc redemande de la mémoire au système.

### Pourquoi déborder fait planter free

Écrire après la fin d'un bloc écrase l'en-tête du suivant. Au prochain free, malloc trouve un en-tête corrompu et arrête le programme (« free(): invalid next size »).

L'ALIGNEMENT

# Le padding dans les structures

Le compilateur place chaque membre à une adresse multiple de sa taille, et ajoute des octets vides (le padding) pour y arriver.

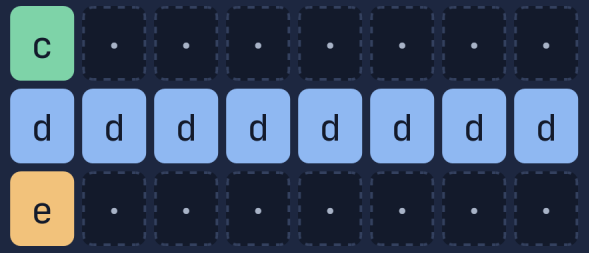
```
struct s1 {
    char c;
    int  i;
};
```




`sizeof = 8`

et pas 5

```
struct s2 {
    char   c;
    double d;
    char   e;
};
```




`sizeof = 24`

mal rangée

```
struct s3 {
    double d;
    char   c;
    char   e;
};
```




`sizeof = 16`

mêmes membres, rangés du plus grand au plus petit

Une case = 1 octet · « · » = padding · la taille totale est arrondie au multiple du plus grand membre

L'ALIGNEMENT

# Aligner une adresse avec un masque

Aligné sur 8 = adresse multiple de 8 = ses 3 derniers bits valent 0. Sur nos machines 64 bits, malloc renvoie même des adresses multiples de 16.

```
  7 = 0000 … 0000 0111
~7 = 1111 … 1111 1000
```

~ inverse tous les bits : & ~7 force les 3 derniers à 0 et garde le reste.

### Arrondir vers le bas

`addr & ~7`

`0x1003 & ~7 = 0x1000`

### Arrondir vers le haut

`(addr + 7) & ~7`

`(0x1003 + 7) & ~7 = 0x1008`

### Tester

`(addr & 7) == 0`

Vrai si l'adresse est alignée sur 8.

POUR FINIR

# Petits tests

## 01

```
10 & -1 ?
```

## 02

```
7 >> 1 ?
```

## 03

```
sizeof(struct {
  char c;
  int  i;
}) ?
```

## 04

```
char *s = "42";
sizeof(s) ?
strlen(s) ?
```

# Merci !

## Des questions ?

42 Mulhouse